Shuhan_Zhang_R_Code_Sample_Firm_Investment_Oil_Shocks

```r
# ============================================================================
# R CODE SAMPLE
# Firm-Level Investment Responses to Oil Price Shocks
# ============================================================================
#
# Purpose
# -------
# This script demonstrates an end-to-end empirical workflow in R. It:
#   1. imports and validates a firm-year panel;
#   2. constructs treatment variables and consecutive-year lags;
#   3. aligns monthly structural oil shocks with firm-specific fiscal years;
#   4. estimates two-way fixed-effects panel models with multiway clustered
#      standard errors;
#   5. evaluates dynamic cumulative effects; and
#   6. conducts selected robustness checks.
#
# Research question
# -----------------
# Do positive oil-price shocks reduce firm-level capital investment, and are
# these effects larger for firms operating in energy-intensive industries?
#
# Data note
# ---------
# The firm-level data are proprietary and therefore are not distributed with
# this code sample. Relative paths below illustrate the intended project
# structure. Variable definitions are summarized where they first appear.
#
# Reproducibility note
# --------------------
# This script assumes one observation per firm-year. All lagged variables are
# constructed only when fiscal years are consecutive, which prevents an
# unbalanced panel from incorrectly treating a multi-year gap as a one-year lag.
# ============================================================================

# ----------------------------------------------------------------------------
# 1. Package setup and project paths
# ----------------------------------------------------------------------------

required_packages <- c(
  "car",
  "data.table",
  "dplyr",
  "fixest",
  "here",
  "knitr",
  "lubridate",
  "readr",
  "readxl",
  "splines",
  "tidyr"
)

missing_packages <- required_packages[
  !vapply(required_packages, requireNamespace, logical(1), quietly = TRUE)
]

if (length(missing_packages) > 0L) {
  stop(
```

```r
    "Install the following packages before running this script: ",
    paste(missing_packages, collapse = ", ")
  )
}

library(car)
library(data.table)
library(dplyr)
library(fixest)
library(here)
library(knitr)
library(lubridate)
library(readr)
library(readxl)
library(splines)
library(tidyr)

# Use relative paths so the analysis is portable across computers.
paths <- list(
  panel = here("data", "processed", "thesis_model_df.csv"),
  controls = here("data", "processed", "panel_patch_controls.csv"),
  supply_shocks = here("data", "raw", "BH2_supply_shocks.xlsx"),
  demand_shocks = here("data", "raw", "BH2_demand_shocks.xlsx"),
  tables = here("output", "tables")
)

dir.create(paths$tables, recursive = TRUE, showWarnings = FALSE)

# ------------------------------------------------------------------------------
# 2. Reusable helper functions
# ------------------------------------------------------------------------------

# Winsorize a numeric vector by replacing values below and above the selected
# quantiles. This limits the influence of extreme accounting observations while
# preserving the sample size.
winsorize_vec <- function(x, lower = 0.01, upper = 0.99) {
  if (!is.numeric(x)) {
    stop("winsorize_vec() requires a numeric vector.")
  }
  if (lower < 0 || upper > 1 || lower >= upper) {
    stop("Choose quantile bounds satisfying 0 <= lower < upper <= 1.")
  }

  bounds <- stats::quantile(
    x,
    probs = c(lower, upper),
    na.rm = TRUE,
    names = FALSE
  )

  pmin(pmax(x, bounds[1]), bounds[2])
}

# Construct a one-period lag only when the current and previous observations
# belong to consecutive years. This is essential for an unbalanced panel.
consecutive_lag <- function(x, year) {
  lagged_x <- dplyr::lag(x)
  lagged_year <- dplyr::lag(year)

  dplyr::if_else(
    !is.na(lagged_year) & year == lagged_year + 1L,
    lagged_x,
```

```r
    NA_real_
  )
}

# Calculate the cumulative effect of a current and lagged coefficient using the
# full variance-covariance matrix. The covariance term is necessary because the
# two estimates are obtained from the same regression.
cumulative_effect <- function(model, current_term, lagged_term) {
  coefficients <- stats::coef(model)
  variance_matrix <- stats::vcov(model)

  required_terms <- c(current_term, lagged_term)
  if (!all(required_terms %in% names(coefficients))) {
    stop("The requested coefficient names are not present in the model.")
  }

  estimate <- coefficients[current_term] + coefficients[lagged_term]
  standard_error <- sqrt(
    variance_matrix[current_term, current_term] +
      variance_matrix[lagged_term, lagged_term] +
      2 * variance_matrix[current_term, lagged_term]
  )

  z_statistic <- estimate / standard_error
  p_value <- 2 * stats::pnorm(abs(z_statistic), lower.tail = FALSE)

  tibble(
    estimate = unname(estimate),
    standard_error = unname(standard_error),
    statistic = unname(z_statistic),
    p_value = unname(p_value)
  )
}

# Stop early when a required variable is missing. Explicit validation makes
# later modeling errors easier to diagnose.
validate_columns <- function(data, required_columns, data_name) {
  missing_columns <- setdiff(required_columns, names(data))

  if (length(missing_columns) > 0L) {
    stop(
      data_name,
      " is missing required columns: ",
      paste(missing_columns, collapse = ", ")
    )
  }
}

# ------------------------------------------------------------------------------
# 3. Import, merge, and validate the firm-year data
# ------------------------------------------------------------------------------

# The Python data-preparation step creates two processed files:
#   1. thesis_model_df.csv contains the main regression-ready panel; and
#   2. panel_patch_controls.csv contains supplemental balance-sheet controls
#      and fiscal year-end dates used for fiscal-year shock alignment.
#
# Some earlier versions of the processed panel did not include datadate,
# cash_at, or tangibility directly. The merge below is therefore written to be
# robust: it validates core columns in the main panel first, then attaches the
# supplemental controls from the patch file.
firm_panel <- read_csv(paths$panel, show_col_types = FALSE)
```

```r
control_patch <- read_csv(paths$controls, show_col_types = FALSE)

validate_columns(
  firm_panel,
  c(
    "gvkey", "fyear", "energy_intensity", "oil_shock",
    "inv_at", "inv_at_w", "inv_ppent_w", "ln_at", "lev", "lev_w",
    "prof", "prof_w", "sales_growth", "sales_growth_w"
  ),
  "firm_panel"
)

validate_columns(
  control_patch,
  c("gvkey", "fyear", "datadate", "cash_at", "tangibility"),
  "control_patch"
)

# Harmonize join keys before combining the processed data sources.
firm_panel <- firm_panel %>%
  mutate(
    gvkey = as.character(gvkey),
    fyear = as.integer(fyear)
  )

# If the main panel already contains datadate, keep it; otherwise create a
# placeholder that will be filled from the patch file after the join.
if ("datadate" %in% names(firm_panel)) {
  firm_panel <- firm_panel %>% mutate(datadate = as.Date(datadate))
} else {
  firm_panel$datadate <- as.Date(NA)
}

control_patch <- control_patch %>%
  mutate(
    gvkey = as.character(gvkey),
    fyear = as.integer(fyear),
    datadate_patch = as.Date(datadate)
  )

# Each input should have one record per firm-year before the merge. This avoids
# accidental many-to-many joins that would duplicate observations.
if (anyDuplicated(firm_panel[c("gvkey", "fyear")]) > 0L) {
  stop("The main panel contains duplicate firm-year observations.")
}

if (anyDuplicated(control_patch[c("gvkey", "fyear")]) > 0L) {
  stop("The control patch contains duplicate firm-year observations.")
}

control_patch_for_join <- control_patch %>%
  select(
    gvkey,
    fyear,
    datadate_patch,
    cash_at,
    tangibility,
    any_of(c("cogs_share", "sga_intensity", "op_margin"))
  )

firm_panel <- firm_panel %>%
  left_join(control_patch_for_join, by = c("gvkey", "fyear")) %>%
```

```r
  mutate(datadate = coalesce(datadate, datadate_patch)) %>%
  select(-datadate_patch)

validate_columns(
  firm_panel,
  c("datadate", "cash_at", "tangibility"),
  "merged firm_panel"
)

# Each record must represent a unique firm-year. Duplicate records would make
# both lag construction and fixed-effects estimation ambiguous.
if (anyDuplicated(firm_panel[c("gvkey", "fyear")]) > 0L) {
  stop("The merged panel contains duplicate firm-year observations.")
}

if (any(is.na(firm_panel$gvkey)) || any(is.na(firm_panel$fyear))) {
  stop("Firm identifiers and fiscal years must not be missing.")
}

if (all(is.na(firm_panel$datadate))) {
  stop("Fiscal year-end dates are missing, so fiscal-year shock alignment cannot be performed.")
}

message(
  "Panel coverage: ",
  min(firm_panel$fyear, na.rm = TRUE),
  "-",
  max(firm_panel$fyear, na.rm = TRUE),
  "; firms: ",
  n_distinct(firm_panel$gvkey),
  "; observations: ",
  nrow(firm_panel)
)

# ------------------------------------------------------------------------------
# 4. Construct treatment variables, controls, and valid lags
# ------------------------------------------------------------------------------

# Define high-energy industries using the 75th percentile of the time-invariant
# industry energy-intensity measure.
high_energy_cutoff <- quantile(
  firm_panel$energy_intensity,
  probs = 0.75,
  na.rm = TRUE,
  names = FALSE
)

firm_panel <- firm_panel %>%
  mutate(
    high_energy = as.integer(energy_intensity >= high_energy_cutoff),

    # Positive-only shocks allow the analysis to focus on oil-price increases,
    # which are the theoretically relevant adverse cost shocks.
    oil_shock_pos = pmax(oil_shock, 0),

    # Winsorize selected accounting ratios at the 1st and 99th percentiles.
    cash_at_w = winsorize_vec(cash_at),
    tangibility_w = winsorize_vec(tangibility)
  ) %>%
  arrange(gvkey, fyear, datadate) %>%
  group_by(gvkey) %>%
  mutate(
```

```r
    oil_shock_pos_l1 = consecutive_lag(oil_shock_pos, fyear),
    ln_at_l1 = consecutive_lag(ln_at, fyear),
    lev_w_l1 = consecutive_lag(lev_w, fyear),
    prof_w_l1 = consecutive_lag(prof_w, fyear),
    sales_growth_w_l1 = consecutive_lag(sales_growth_w, fyear),
    cash_at_w_l1 = consecutive_lag(cash_at_w, fyear),
    tangibility_w_l1 = consecutive_lag(tangibility_w, fyear)
  ) %>%
  ungroup() %>%
  mutate(
    oil_x_high = oil_shock * high_energy,
    oil_pos_x_high = oil_shock_pos * high_energy,
    oil_pos_x_high_l1 = oil_shock_pos_l1 * high_energy
  )

# ------------------------------------------------------------------------------
# 5. Import and clean monthly structural oil shocks
# ------------------------------------------------------------------------------

# The source spreadsheets contain two descriptive header rows and do not use
# analysis-ready column names, so the relevant fields are selected explicitly.
supply_raw <- read_excel(paths$supply_shocks, col_names = FALSE)
demand_raw <- read_excel(paths$demand_shocks, col_names = FALSE)

supply_shocks <- supply_raw %>%
  slice(-(1:2)) %>%
  select(1, 2) %>%
  rename(
    date = ...1,
    oil_supply_shock = ...2
  ) %>%
  mutate(
    date = as.Date(date),
    oil_supply_shock = as.numeric(oil_supply_shock)
  )

demand_shocks <- demand_raw %>%
  slice(-(1:2)) %>%
  select(1, 2, 3, 4) %>%
  rename(
    date = ...1,
    economic_activity_shock = ...2,
    oil_consumption_demand_shock = ...3,
    oil_inventory_demand_shock = ...4
  ) %>%
  mutate(
    date = as.Date(date),
    across(
      c(
        economic_activity_shock,
        oil_consumption_demand_shock,
        oil_inventory_demand_shock
      ),
      as.numeric
    )
  )

monthly_shocks <- supply_shocks %>%
  left_join(demand_shocks, by = "date") %>%
  arrange(date)

if (anyDuplicated(monthly_shocks$date) > 0L) {
```

```r
  stop("The monthly shock data contain duplicate dates.")
}

# ------------------------------------------------------------------------------
# 6. Align monthly shocks with firm-specific fiscal years
# ------------------------------------------------------------------------------

# A calendar-year aggregation would misclassify exposure for firms whose fiscal
# years do not end in December. Instead, each firm-year is assigned the monthly
# shocks observed during its own 12-month reporting window.
firm_fiscal_windows <- firm_panel %>%
  mutate(
    fiscal_end_month = floor_date(datadate, unit = "month"),
    fiscal_start_month = fiscal_end_month %m-% months(11)
  )

firm_dt <- as.data.table(firm_fiscal_windows)
shock_dt <- as.data.table(monthly_shocks)

# Non-equijoin: match each monthly observation to every firm-year fiscal window
# that contains that month.
matched_months <- shock_dt[
  firm_dt,
  on = .(
    date >= fiscal_start_month,
    date <= fiscal_end_month
  ),
  allow.cartesian = TRUE,
  nomatch = 0L
]

# Confirm that most firm-years are matched to 12 monthly observations. Keeping
# this check visible documents the temporal alignment and reveals incomplete
# windows near the boundaries of the shock series.
match_counts <- matched_months[, .N, by = .(gvkey, fyear)]
message(
  "Fiscal-year monthly matches: median = ",
  stats::median(match_counts$N),
  "; range = ",
  min(match_counts$N),
  "-",
  max(match_counts$N)
)

fiscal_shocks <- matched_months[
  ,
  .(
    oil_supply_shock_fy = mean(oil_supply_shock, na.rm = TRUE),
    economic_activity_shock_fy = mean(
      economic_activity_shock,
      na.rm = TRUE
    ),
    oil_consumption_demand_shock_fy = mean(
      oil_consumption_demand_shock,
      na.rm = TRUE
    ),
    oil_inventory_demand_shock_fy = mean(
      oil_inventory_demand_shock,
      na.rm = TRUE
    ),
    matched_months = .N
  ),
```

```r
  by = .(gvkey, fyear)
]

firm_panel <- firm_panel %>%
  left_join(as_tibble(fiscal_shocks), by = c("gvkey", "fyear")) %>%
  arrange(gvkey, fyear, datadate) %>%
  group_by(gvkey) %>%
  mutate(
    supply_x_high_fy = oil_supply_shock_fy * high_energy,
    activity_x_high_fy = economic_activity_shock_fy * high_energy,
    supply_x_high_fy_l1 = consecutive_lag(supply_x_high_fy, fyear),
    activity_x_high_fy_l1 = consecutive_lag(activity_x_high_fy, fyear)
  ) %>%
  ungroup()

# ------------------------------------------------------------------------------
# 7. Define common analysis samples
# ------------------------------------------------------------------------------

# Using a common sample ensures coefficient changes across specifications are
# driven by the model rather than by different missing-value patterns.
control_variables <- c(
  "ln_at_l1",
  "lev_w_l1",
  "prof_w_l1",
  "sales_growth_w_l1",
  "cash_at_w_l1",
  "tangibility_w_l1"
)

reduced_form_sample <- firm_panel %>%
  drop_na(
    inv_at_w,
    oil_x_high,
    oil_pos_x_high,
    oil_pos_x_high_l1,
    all_of(control_variables)
  )

decomposition_sample <- firm_panel %>%
  drop_na(
    inv_at_w,
    supply_x_high_fy,
    supply_x_high_fy_l1,
    activity_x_high_fy,
    activity_x_high_fy_l1,
    all_of(control_variables)
  )

message(
  "Reduced-form sample: ", nrow(reduced_form_sample), " observations; ",
  n_distinct(reduced_form_sample$gvkey), " firms."
)
message(
  "Decomposition sample: ", nrow(decomposition_sample), " observations; ",
  n_distinct(decomposition_sample$gvkey), " firms."
)

# ------------------------------------------------------------------------------
# 8. Estimate two-way fixed-effects models
# ------------------------------------------------------------------------------

```

```r
# All models include firm and fiscal-year fixed effects. Standard errors are
# clustered by both firm and year to allow arbitrary serial correlation within
# firms and common shocks within years.
model_baseline <- feols(
  inv_at_w ~ oil_x_high +
    ln_at_l1 + lev_w_l1 + prof_w_l1 + sales_growth_w_l1 +
    cash_at_w_l1 + tangibility_w_l1 |
    gvkey + fyear,
  data = reduced_form_sample,
  vcov = ~ gvkey + fyear
)

model_positive_shock <- feols(
  inv_at_w ~ oil_pos_x_high +
    ln_at_l1 + lev_w_l1 + prof_w_l1 + sales_growth_w_l1 +
    cash_at_w_l1 + tangibility_w_l1 |
    gvkey + fyear,
  data = reduced_form_sample,
  vcov = ~ gvkey + fyear
)

# The dynamic specification includes current and one-year-lagged exposure. This
# captures delayed capital-budget responses and permits a cumulative-effect test.
model_dynamic <- feols(
  inv_at_w ~ oil_pos_x_high + oil_pos_x_high_l1 +
    ln_at_l1 + lev_w_l1 + prof_w_l1 + sales_growth_w_l1 +
    cash_at_w_l1 + tangibility_w_l1 |
    gvkey + fyear,
  data = reduced_form_sample,
  vcov = ~ gvkey + fyear
)

# Structural decomposition separates supply-driven oil shocks from shocks tied
# to aggregate economic activity. The distinction helps identify whether higher
# oil prices reflect adverse cost pressure or stronger macroeconomic demand.
model_supply_shock <- feols(
  inv_at_w ~ supply_x_high_fy + supply_x_high_fy_l1 +
    ln_at_l1 + lev_w_l1 + prof_w_l1 + sales_growth_w_l1 +
    cash_at_w_l1 + tangibility_w_l1 |
    gvkey + fyear,
  data = decomposition_sample,
  vcov = ~ gvkey + fyear
)

model_activity_shock <- feols(
  inv_at_w ~ activity_x_high_fy + activity_x_high_fy_l1 +
    ln_at_l1 + lev_w_l1 + prof_w_l1 + sales_growth_w_l1 +
    cash_at_w_l1 + tangibility_w_l1 |
    gvkey + fyear,
  data = decomposition_sample,
  vcov = ~ gvkey + fyear
)

# ------------------------------------------------------------------------------
# 9. Evaluate cumulative current-plus-lagged effects
# ------------------------------------------------------------------------------

cumulative_results <- bind_rows(
  cumulative_effect(
    model_dynamic,
    "oil_pos_x_high",
    "oil_pos_x_high_l1"
```

```r
  ) %>%
    mutate(model = "Positive oil-price shock"),

  cumulative_effect(
    model_supply_shock,
    "supply_x_high_fy",
    "supply_x_high_fy_l1"
  ) %>%
    mutate(model = "Structural oil-supply shock"),

  cumulative_effect(
    model_activity_shock,
    "activity_x_high_fy",
    "activity_x_high_fy_l1"
  ) %>%
    mutate(model = "Economic-activity shock")
) %>%
  select(model, everything())

print(cumulative_results)

# Cross-check the manually calculated variance of the coefficient sum using a
# formal linear hypothesis test.
print(
  linearHypothesis(
    model_dynamic,
    "oil_pos_x_high + oil_pos_x_high_l1 = 0",
    vcov. = vcov(model_dynamic)
  )
)

# ------------------------------------------------------------------------------
# 10. Selected robustness checks
# ------------------------------------------------------------------------------

# Robustness check 1: use capital expenditure divided by beginning property,
# plant, and equipment as an alternative investment measure.
alt_outcome_sample <- firm_panel %>%
  drop_na(
    inv_ppent_w,
    oil_pos_x_high,
    oil_pos_x_high_l1,
    all_of(control_variables)
  )

model_alt_outcome <- feols(
  inv_ppent_w ~ oil_pos_x_high + oil_pos_x_high_l1 +
    ln_at_l1 + lev_w_l1 + prof_w_l1 + sales_growth_w_l1 +
    cash_at_w_l1 + tangibility_w_l1 |
    gvkey + fyear,
  data = alt_outcome_sample,
  vcov = ~ gvkey + fyear
)

# Robustness check 2: replace the high-energy indicator with a flexible natural
# spline in continuous energy intensity. This allows heterogeneous effects to
# vary nonlinearly rather than imposing a single threshold.
spline_basis <- ns(firm_panel$energy_intensity, df = 3)

spline_panel <- firm_panel %>%
  mutate(
    spline_1 = spline_basis[, 1],
```

```r
    spline_2 = spline_basis[, 2],
    spline_3 = spline_basis[, 3],
    oil_pos_x_spline_1 = oil_shock_pos * spline_1,
    oil_pos_x_spline_2 = oil_shock_pos * spline_2,
    oil_pos_x_spline_3 = oil_shock_pos * spline_3
  ) %>%
  drop_na(
    inv_at_w,
    oil_pos_x_spline_1,
    oil_pos_x_spline_2,
    oil_pos_x_spline_3,
    all_of(control_variables)
  )

model_spline <- feols(
  inv_at_w ~ oil_pos_x_spline_1 + oil_pos_x_spline_2 +
    oil_pos_x_spline_3 +
    ln_at_l1 + lev_w_l1 + prof_w_l1 + sales_growth_w_l1 +
    cash_at_w_l1 + tangibility_w_l1 |
    gvkey + fyear,
  data = spline_panel,
  vcov = ~ gvkey + fyear
)

# Jointly test whether the three nonlinear interaction terms are zero.
print(wald(model_spline, keep = "oil_pos_x_spline"))

# ------------------------------------------------------------------------------
# 11. Export presentation-ready results
# ------------------------------------------------------------------------------

variable_labels <- c(
  "oil_x_high" = "Oil shock x high energy intensity",
  "oil_pos_x_high" = "Positive oil shock x high energy intensity",
  "oil_pos_x_high_l1" = "Lagged positive oil shock x high energy intensity",
  "supply_x_high_fy" = "Supply shock x high energy intensity",
  "supply_x_high_fy_l1" = "Lagged supply shock x high energy intensity",
  "activity_x_high_fy" = "Activity shock x high energy intensity",
  "activity_x_high_fy_l1" = "Lagged activity shock x high energy intensity",
  "ln_at_l1" = "Firm size",
  "lev_w_l1" = "Leverage",
  "prof_w_l1" = "Profitability",
  "sales_growth_w_l1" = "Sales growth",
  "cash_at_w_l1" = "Cash holdings",
  "tangibility_w_l1" = "Tangibility"
)

etable(
  list(
    "Baseline" = model_baseline,
    "Positive shock" = model_positive_shock,
    "Dynamic positive shock" = model_dynamic
  ),
  dict = variable_labels,
  tex = TRUE,
  replace = TRUE,
  digits = 4,
  file = file.path(paths$tables, "reduced_form_models.tex")
)

etable(
  list(
```

```r
    "Supply shock" = model_supply_shock,
    "Activity shock" = model_activity_shock,
    "Alternative outcome" = model_alt_outcome
  ),
  dict = variable_labels,
  tex = TRUE,
  replace = TRUE,
  digits = 4,
  file = file.path(paths$tables, "decomposition_and_robustness.tex")
)

write_csv(
  cumulative_results,
  file.path(paths$tables, "cumulative_effects.csv")
)

# Session information records the R version and package environment used to run
# the analysis, improving reproducibility for reviewers.
writeLines(
  capture.output(sessionInfo()),
  here("output", "session_info.txt")
)
```